

P2769R0

get_element customization point object

Published Proposal, 2022-01-13

Authors:

[Ruslan Arutyunyan](#) (Intel)

[Alexey Kukanov](#) (Intel)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper introduces a CPO to read from tuple like objects, which can be used as projection for `std::ranges` algorithms.

Table of Contents

1 Motivation

- 1.1 Motivating use-case
- 1.2 Projections-based alternative
- 1.3 The desired approach
- 1.4 Why not `std::ranges::views::elements`
- 1.5 Radix sort use-case

2 Proposed API

- 2.1 Possible implementation

3 Design considerations

- 3.1 What could be done to use `std::ranges::get` name

4 Further steps

§ 1. Motivation

§ 1.1. Motivating use-case

Having `std::pair`, `std::tuple` and other *Tuple-Like* objects as the value types for the algorithms creates a plenty of opportunities. With special views (like `std::ranges::elements_view`) we can even iterate over such collections with specifying necessary elements (e.g., keys or values). However, we cannot (easily) use a predicate to make a decision based on (for example) keys.

Let's consider the following example:

```
std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}};
std::ranges::sort(v, [](auto x, auto y)
{
    return std::get<0>(x) < std::get<0>(y); // key-based sorting
});
```

As we can see users should do some extra syntax to spell (comparing with what is described in § 1.3 The desired approach) to achieve the necessary goal. The example above can be considered as simplified because in real world example users might need to think about adding references to lambda signature to avoid copying.

The code above can be rewritten with *structured binding*:

```
std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}};
std::ranges::sort(v, [](auto x, auto y)
{
    auto [key1, value1] = x;
    auto [key2, value2] = y;
    return key1 < key2; // key-based sorting
});
```

Though one could say that it makes code simpler or at least more readable, on the other hand, its syntax forces the programmer to give names to otherwise unneeded variables, which is often considered a bad practice.

§ 1.2. Projections-based alternative

Projections provide another option to achieve the same behavior:

```
std::ranges::sort(v, std::less{}, [](auto x)
{
    return std::get<0>(x); // key-based sorting
});
```

A variant that properly handles references would use a generic lambda:

```
[](auto&& x) -> auto&&
{
    return std::get<0>(std::forward<decltype(x)>(x)); // key-based sorting
}
```

While this code achieves the desired result, it requires more syntactic boilerplate (lambda, forwarding etc.) than the useful code.

§ 1.3. The desired approach

The nicest way to get what we want would be:

```
// The code that does not work because std::get is not fully instantiated
std::ranges::sort(v, std::less{}, std::get<0>);
```

But it doesn't work because `std::get` is a function template, and one cannot pass function templates as arguments without instantiating them.

§ 1.4. Why not `std::ranges::views::elements`

The necessary result cannot be achieved with `std::ranges::views::elements`, which would apply the filter for all operations on the input data, including element swap (for sort algorithm), while we need it to be only be applied for the comparator.

std::ranges::views::elements		Des
<pre>std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}}; // views::keys is an alias to views::elements std::ranges::sort(v std::ranges::views::keys, [](auto x, auto y) { return x < y; });</pre>		<pre>std::vector<std::tuple<int, int> std::ranges::sort(v, [](auto x, { return std::get<0>(x) < std }); for (auto& x : v)</pre>

```

for (auto& x : v)
{
    auto [key, val] = x;
    std::cout << "Key = " << key << ", Value = " << val << std::endl;
}

// Output (only keys are sorted):
// Key = 1, Value = 1
// Key = 2, Value = 4
// Key = 3, Value = 7

```

```

{
    auto [key, val] = x;
    std::cout << "Key = " << key << ", Value = " << val << std::endl;
}

// Output (values are sorted by key):
// Key = 1, Value = 7
// Key = 2, Value = 4
// Key = 3, Value = 1

```

With `std::ranges::zip_view` appearance in the standard the easy use of projection for *TupleLike* objects might become even more important because its dereferenceable type is exactly *TupleLike*.

§ 1.5. Radix sort use-case

Counting-based sorts, and Radix Sort in particular, provide another motivating use case. Today it is not possible to have a C++ standard conformant implementation that uses Radix Sort algorithm underneath because the complexity of `std::sort` is defined as the number of comparator calls, while counting-based sorts do not use a comparator at all.

However, the industry needs Radix Sort for performance reasons. Implementations of C++ standard parallel algorithms, such as oneAPI Data Parallel C++ Library (oneDPL) and CUDA Thrust, use Radix Sort conditionally under the hood of `std::sort`, checking data types of the input and the comparator. In this case, a special comparator is of no help to sort values by keys, and projections seem the only viable option.

In that case the proposed API becomes wider applicable than just with C++ standard library use-cases.

§ 2. Proposed API

We propose the following API:

```

namespace ranges {
    inline namespace /* unspecified */ {
        template <size_t I>
        inline constexpr /* unspecified */ get_element = /* unspecified */;
    }
    inline constexpr auto get_key = get_element<0>;
    inline constexpr auto get_value = get_element<1>;
}

```

With that API the motivating use-case code with the desired behavior would be:

```

std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}};
std::ranges::sort(v, std::less{}, std::ranges::get_element<0>);

```

or even

```

std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}};
std::ranges::sort(v, std::less{}, std::ranges::get_key);

```

Let's look at Tony Tables:

Comparison of proposed API with comparator-based version

Before	After
<pre>std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}}; std::ranges::sort(v, [](auto x, auto y) { return std::get<0>(x) < std::get<0>(y); // key-based sorting });</pre>	<pre>std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}}; std::ranges::sort(v, std::less{}, std::ranges::get<0>);</pre>

Comparison of proposed API with projections-based version

Before	After
<pre>std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}}; std::ranges::sort(v, std::less{}, [](auto x) { return std::get<0>(x); // key-based sorting });</pre>	<pre>std::vector<std::tuple<int, int>> v{{3,1},{2,4},{1,7}}; std::ranges::sort(v, std::less{}, std::ranges::get<0>);</pre>

§ 2.1. Possible implementation

```
namespace std
{
    namespace ranges
    {
        namespace __detail
        {
            template <std::size_t _Ip>
            struct __get_element_fn
            {
                template <typename _TupleLike>
                decltype(auto) operator()(_TupleLike&& __tuple_like) const
                {
                    return get<_Ip>(std::forward<_TupleLike>(__tuple_like));
                }
            };
        } // namespace __detail

        inline namespace __get_element_namespace
        {
            template <std::size_t _Ip>
            inline constexpr __detail::__get_element_fn<_Ip> get_element;
        } // inline namespace __get_element_namespace

        inline constexpr auto get_key = get_element<0>;
        inline constexpr auto get_value = get_element<1>;
    } // namespace ranges
} // namespace std
```

§ 3. Design considerations

We would actually prefer to use the `std::ranges::get` name instead of the proposed one. Unfortunately, this name is already taken for overloads for `std::ranges::subrange`.

Potentially `std::ranges::get` could be repurposed for the proposed CPO with minimal API break for tricky scenarios only while still working as expected in existing reasonable use cases, as explained below. But we (likely) could not avoid an ABI break.

§ 3.1. What could be done to use `std::ranges::get` name

In all major standard library implementations (GCC, LLVM, Microsoft) the `get` overload for `std::ranges::subrange` is defined in `std::ranges`. Adding another definition of `get` to the same namespace would obviously create name ambiguity.

However, library implementors could move both `std::ranges::get` and `std::ranges::subrange` to some implementation specific namespace (e.g., `detail`). That way `std::ranges::get` can still be found by ADL for `std::ranges::subrange`. At the same time, one could then use `std::ranges::get` name for a CPO that could successfully find the necessary overload for `std::ranges::get`, including the case when the argument is a `std::ranges::subrange`.

Please see the example that explains the idea and shows how it might look like:

```
namespace std::ranges
{
    // Moving std::ranges::subrange and std::ranges::get to __detail namespace
    namespace __detail
    {
        // Pseudo subrange
        template <typename T>
        struct subrange {};

        template <std::size_t _Ip, typename T>
        auto get(subrange<T>);
    }
    // Make the std::ranges::subrange name accessible from std::ranges namespace
    using __detail::subrange;
}

// Possible implementation of std::ranges::get CPO
namespace std
{
    namespace ranges
    {
        namespace __detail
        {
            template <std::size_t _Ip>
            struct __get_fn
            {
                template <typename _TupleLike>
                decltype(auto) operator()(_TupleLike&& __tuple_like) const
                {
                    return get<_Ip>(std::forward<_TupleLike>(__tuple_like));
                }
            };
        } // namespace __detail

        inline namespace __get_fn_namespace
        {
            template <std::size_t _Ip>
            inline constexpr __detail::__get_fn<_Ip> get;
        } // inline namespace __get_fn_namespace
    } // namespace ranges
    using ranges::__detail::get;
} // namespace std
```

With such an implementation, three major use-cases continue working: `std::ranges::get<0>(sub_r)`, `std::get<0>(sub_r)` and `get<0>(sub_r)` - where `sub_r` is `std::ranges::subrange` object.

Definitely that would break the ABI because the fully qualified names become different comparing with what we have right now. For more tricky scenarios (e.g., explicit template instantiation of `std::ranges::get`) the API is also broken. Since

the `std::ranges::subrange` API is relatively new, perhaps only a small amount of users would be affected but it can not be predicted accurately.

We would like to hear the opinion of the audience whether going with the `std::ranges::get` name is worth introducing potential inconvenience for some C++ users.

§ 4. Further steps

- Add wording
- Add a feature test macro if necessary